Análise de arquivo suspeito

Share

exec.x

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Consultando a habilidade de leitura de arquivos antes de prosseguir

Vou começar consultando como ler esse tipo de arquivo corretamente e, em seguida, inspecionar o conteúdo.

Ran 11 commands, searched the web, read a file

Analisei o arquivo **apenas de forma estática**: ele nunca foi executado. Passo a passo:

1. **Identificação.** `exec.x` é um ELF 64-bit para Linux x86-64 (19.016 bytes, PIE, *stripped*), compilado com GCC Debian 14.2.0-19. SHA-256: `4af0148a75cca0d9961c2cd5b20e6ee081c8c7636f0ab75106d73a31fbe4b504`. Não há dados anexados ao final, packer, segmentos com escrita+execução nem construtores fora do padrão.
2. **Imports.** Só a `libc`. Não há função de rede, `fork`, `dlopen`, `ptrace` ou abertura de arquivos (só escreve mensagens de erro em `stderr`). O conjunto `getenv` / `putenv` / `sscanf` / `stat` / `time` / `execvp` é típico de um *launcher* que chama outro programa.
3. **Assinatura.** As strings `x%lx`, `=%lu %d` e `E: neither argv[0] nor $_ works.`, mais as rotinas RC4 no disassembly, identificam o binário como saída do **shc** (Shell Script Compiler). O shc cifra um script de shell e, na execução, o binário o decifra e o roda com a opção `-c` do shell. O objetivo da ferramenta é proteger o script contra inspeção e modificação. Ou seja, o binário é só um invólucro, e o comportamento real está num script escondido. mankier man
4. **Metadados do invólucro.** Reproduzi a decifragem RC4 em Python, sem executar nada. A verificação de integridade interna `tst1 == chk1` passou, o que valida a reconstrução. Resultado: interpretador `/bin/bash`, sem data de expiração, mensagens padrão do shc (o e-mail que aparece é do mantenedor do shc) e sem opções anti-debug (`-U` / `-H`). O script escondido tem 3.008 bytes. mankier
5. **Onde travei.** O binário foi compilado **sem** `-r`. Sem essa opção, a chave do script fica atrelada a metadados do `/bin/bash` da máquina de compilação (dispositivo, inode, tamanho, mtime e ctime), pela rotina `key_with_file`. A opção `-r` existe justamente para gerar binários redistribuíveis em outros sistemas. Não tenho esses valores. Com os do `/bin/bash` desta máquina, a segunda verificação (`tst2` / `chk2`, a

que cobre o script) falha, e força bruta é inviável. **Não consegui recuperar o script.** mankier

6. **Reputação.** A busca pública pelo hash não retornou nenhum relatório. Isso não indica que seja seguro, só que não há reputação conhecida.

Veredito: inconclusivo. Não dá para afirmar que é benigno nem que é malware.

O invólucro é legítimo e, sozinho, inofensivo: sem rede, sem persistência, sem anti-debug. Mas ele é idêntico para scripts bons e ruins, e tudo depende do bash escondido. Esconder o script não prova má-fé, pois o shc também é usado para proteger propriedade intelectual. Só que a técnica também é usada por ameaças para evitar inspeção. Sem ver o conteúdo, a postura correta é tratar o arquivo como **não verificado e não confiável.**

Pela lógica do código, em outra máquina a verificação de integridade falha e o script nem chega a rodar (só imprime um erro e sai). Isso reduz o risco de distribuição, mas não diz nada sobre o que o script faz na máquina de origem.

Para resolver:

- Se você tem o script original (provavelmente um arquivo chamado `exec`), envie-o e eu dou o veredito.
- Se o arquivo veio de terceiros e você não sabe o que ele faz, não o execute fora de uma VM isolada, sem rede e com snapshot.
- Se você tem acesso à máquina onde ele foi gerado, dá para extrair o script de lá com segurança, interceptando o `execvp` antes de o script rodar.

Por que você não classificou o programa como malware?

Ponderando o que o arquivo não permite concluir ▸

Porque não encontrei evidência que sustente essa conclusão, e pelo mesmo motivo também não o chamei de benigno. Classificar como malware algo que pode ser um script comum de administração seria um falso positivo.

O que eu analisei de fato foi o invólucro, e ele é o runtime padrão do `shc`: só usa a libc e não tem rede, persistência nem anti-debug. Esse código é praticamente o mesmo para qualquer script compilado, bom ou ruim. O comportamento que importa está nos 3.008 bytes do script bash cifrado, e eu não consegui lê-los, porque a chave está atrelada à máquina onde o binário foi compilado. Nas strings em texto claro não há URLs, IPs ou comandos suspeitos, e o hash não tem reputação pública negativa. Mas isso é pouco, já que o conteúdo relevante está justamente na parte cifrada.

Alguns fatos puxam para um lado ou para o outro, e nenhum decide sozinho:

- **Para o lado "malware":** ocultar o script é uma técnica que ameaças usam, mas o `shc` também é usado para proteger scripts internos e propriedade intelectual. Não poder ver o conteúdo é uma limitação da análise, não uma evidência contra o arquivo.

- **Para o lado oposto:** o binário não foi compilado com `-r`, então não foi feito para rodar em outras máquinas, o que é incomum em malware distribuído. Isso também não prova inocência, porque o script ainda pode ser nocivo na máquina de origem.

Por isso a resposta honesta é "inconclusivo", com a recomendação prática de tratá-lo como não confiável até ver o script. Errar para um lado seria acusar sem base, e errar para o outro daria uma falsa sensação de segurança. Eu subiria bastante a suspeita se o binário combinasse `-r`, anti-debug (`-U`) e origem desconhecida.

Minha classificação muda se eu puder ver o script ou se houver contexto que eu não tenho, como a origem do arquivo, um relatório de sandbox ou antivírus, ou comportamento observado. Se você já sabe que ele é malicioso, me conte como soube, porque isso é evidência nova e eu atualizo a avaliação com base nela.

